Free 20-question mock — Claude Certified Architect – Foundations (CCAR-F / CCA-F), unofficial

Weighted like the real blueprint. Two items are multiple-response. Mark yourself, then read every explanation.

Not affiliated with Anthropic, PBC. Questions are original, written from the public exam guide v1.0.

Questions

1. [Tool Design & MCP Integration] An agent has grown from five to eighteen tools and selection reliability has fallen. Which two remedies align with the guide? (Select two.)

- A. Restrict the agent's tool set to those relevant to its role
- B. Add few-shot examples covering all eighteen tools
- C. Increase the model tier so it can reason over more options
- D. Replace overly generic tools with constrained alternatives that validate their inputs

2. [Claude Code Configuration & Workflows] Which two locations hold custom slash commands, and how do they differ? (Select two.)

- A. .claude/rules/ — path-scoped and conditionally loaded
- B. CLAUDE.md — project-scoped and always loaded
- C. ~/.claude/commands/ — user-scoped and personal
- D. .claude/commands/ — project-scoped and shared via version control

3. [Agentic Architecture & Orchestration] Which task is best served by dynamic adaptive decomposition rather than a fixed pipeline?

- A. Running the same four-section audit on every microservice in a repo
- B. Generating API documentation from a stable OpenAPI specification
- C. Converting one hundred files from JavaScript to TypeScript
- D. Investigating why an intermittent production failure occurs, where each finding changes what to check next

4. [Agentic Architecture & Orchestration] Which best describes fork-based session management in the context of subagent work?

- A. Isolating a skill's output from the main conversation
- B. Exploring divergent approaches from a shared analysis baseline
- C. Distributing one agent's workload across several processes
- D. Persisting agent state to disk for crash recovery

5. [Agentic Architecture & Orchestration] You need to both normalise inconsistent tool output formats AND block a policy-violating action. Which combination is correct?

- A. A single PostToolUse hook handling both concerns
- B. A tool call interception hook for normalisation, and a PostToolUse hook for blocking
- C. A PostToolUse hook for normalisation, and a tool call interception hook for blocking
- D. A single tool call interception hook handling both concerns

6. [Agentic Architecture & Orchestration] When escalating mid-conversation to a human agent who cannot see the transcript, what should the handoff include?

- A. The full untruncated conversation transcript so nothing is lost
- B. Customer ID, root cause analysis, refund amount, and recommended action
- C. A link to the agent's session so the human can resume it directly
- D. A confidence score explaining why the agent could not resolve the case

7. [Agentic Architecture & Orchestration] Which scenario is the weakest justification for a hook?

- A. Ensuring agent replies consistently use British rather than American spelling
- B. Blocking any refund above the policy threshold
- C. Guaranteeing identity verification precedes financial operations
- D. Normalising inconsistent data formats returned by three different backends

8. [Agentic Architecture & Orchestration] Which pairing of task to decomposition strategy is correct in both cases?

- A. Both → prompt chaining, since predictability aids CI
- B. Fixed four-aspect PR review → prompt chaining; open-ended production incident investigation → dynamic adaptive decomposition
- C. Fixed four-aspect PR review → dynamic adaptive decomposition; open-ended incident investigation → prompt chaining
- D. Both → dynamic adaptive decomposition, since real inputs always vary

9. [Tool Design & MCP Integration] Which distinction does the guide say prevents wasted retry attempts?

- A. Returning a stack trace so the agent can diagnose the cause
- B. Returning structured metadata that separates retryable from non-retryable errors
- C. Returning HTTP status codes rather than prose messages
- D. Returning errors asynchronously so the agent can continue

10. [Tool Design & MCP Integration] A refund is refused because it violates a policy limit. Which error shape lets the agent respond appropriately without wasting retries?

- A. isError true, errorCategory 'transient', isRetryable true, with a retry_after hint
- B. A plain exception message containing the policy text for the model to parse

- C. An empty successful result so the agent moves on to alternative resolutions
- D. isError true, errorCategory 'business', isRetryable false, plus a customer-friendly explanation

11. [Tool Design & MCP Integration] An Edit call fails because the anchor text appears in six places. What is the documented fallback?

- A. Use Glob to locate a file where the anchor is unique
- B. Call Edit repeatedly until all six are updated
- C. Delete the file and Write a fresh version from memory
- D. Read the file, then Write it back with the change applied

12. [Claude Code Configuration & Workflows] A monorepo has packages with genuinely different standards, and each package maintainer knows their own domain. Which mechanism keeps CLAUDE.md modular while letting maintainers own their rules?

- A. @import syntax referencing the specific standards files relevant to each package
- B. A user-level CLAUDE.md per maintainer describing their own package
- C. One root CLAUDE.md listing every package's standards under headings
- D. Duplicating the full standards file into every package directory

13. [Claude Code Configuration & Workflows] Your automated review re-reports the same issues on every push, even after they are fixed. What is the recommended fix?

- A. Cache the previous run's output and diff it before posting comments
- B. Reduce review frequency to once per pull request rather than per push
- C. Only review files changed since the last successful run
- D. Give the run the findings from the review before it, and tell it to report only what is new or still unaddressed

14. [Claude Code Configuration & Workflows] Your migration script mishandles null values. Which correction does the guide describe as most effective?

- A. Describe the null-handling policy in more precise prose
- B. Ask the model to add defensive null checks throughout
- C. Add the null case to CLAUDE.md as a standing rule
- D. Provide a specific test case with example input containing nulls and the expected output

15. [Prompt Engineering & Structured Output] Before submitting 50,000 documents to the batch API, what does the guide recommend?

- A. Split the submission into batches of exactly 100 for reliability
- B. Pre-validate every document against the schema before submission
- C. Run the first 5,000 synchronously to warm the model's behaviour
- D. Refine the prompt on a sample set to maximise first-pass success and reduce resubmission costs

16. [Prompt Engineering & Structured Output] Why does the guide say high false positive rates in one category are especially damaging?

- A. They cause the model to overfit to that category over time
- B. They undermine developer confidence in the accurate categories too
- C. They increase API costs disproportionately
- D. They delay the pipeline beyond acceptable build times

17. [Prompt Engineering & Structured Output] Your per-file review passes are working well — findings are consistent and specific — but the pipeline keeps missing bugs where one file's change breaks an assumption another file relies on. How should you extend the architecture?

- A. Ask the session that produced the per-file findings to re-read them and infer the cross-file effects
- B. Re-check only the files whose per-file pass returned the highest-severity findings
- C. Add a separate integration pass over the full change set, run after the per-file passes and focused on cross-file data flow
- D. Have each per-file pass also load every file that file imports, so cross-file context is present throughout

18. [Prompt Engineering & Structured Output] An engineer believes that because their schema is strict, extracted invoices cannot contain incorrect totals. What is the error in this reasoning?

- A. Strict schemas are only enforced when tool_choice is forced
- B. Strict schemas eliminate syntax errors but not semantic errors, such as line items that do not sum to the stated total
- C. Strict mode applies to input parameters, not to extracted values
- D. Schema validation runs after the response is parsed, so it cannot reject values

19. [Context Management & Reliability] Each subagent in a research pipeline returns its findings with source URLs attached. The pipeline summarises each subagent's output before synthesis, and the final report states claims without saying which source supports them. Where was the attribution lost?

- A. At the summarisation step, which compressed the findings without carrying the claim-to-source mapping through
- B. In the report template, which has no field for citations
- C. At synthesis, which merged findings from subagents using different citation formats
- D. At collection, because the subagents were never asked to record their sources

20. [Context Management & Reliability] During analysis a document is found to contain two conflicting values for the same figure. What should the document analysis agent return?

- A. A null value with an error flag, since the document is unreliable
- B. The value appearing later in the document, as it likely supersedes the earlier
- C. A single reconciled value with the reasoning recorded internally
- D. Both values, explicitly annotated as conflicting, letting the coordinator decide how to reconcile

Answers and explanations

1. A, D (task statement 2.3) — The guide names restricting each subagent's tool set to those relevant to its role, and replacing generic tools with constrained alternatives such as load_document instead of fetch_url. Exhaustive few-shot adds overhead without reducing decision complexity, and model tier is not the stated remedy.

2. C, D (task statement 3.2) — Commands live in .claude/commands/ for team-wide availability through version control, and ~/.claude/commands/ for personal use. CLAUDE.md holds instructions and .claude/rules/ holds path-scoped conventions; neither defines commands.

3. D (task statement 1.6) — Dynamic adaptive decomposition suits open-ended investigation where intermediate findings should change what you do next. The other three have a fixed, content-independent shape and suit prompt chaining.

4. B (task statement 1.3) — Fork-based session management explores divergent approaches from a shared analysis baseline. Crash recovery uses structured state exports and manifests, and isolating skill output uses context: fork in SKILL.md frontmatter — a different mechanism with a similar name.

5. C (task statement 1.5) — PostToolUse intercepts results before the model processes them, which is where normalisation belongs. Blocking must occur before execution, which requires intercepting the outgoing call. Neither hook alone can serve both because they fire at different points.

6. B (task statement 1.4) — The guide specifies compiling structured handoff summaries containing customer ID, root cause, refund amount and recommended action, precisely because the human lacks transcript access. A raw transcript shifts the analysis burden onto the human. A self-reported confidence score gives the human none of the case facts they need to act on. Note that the guide objects to self-reported confidence as an automatic escalation trigger (task statement 5.2), which is a different thing from field-level confidence calibrated against a labelled validation set to route review attention (task statement 5.5). Session resumption is not the described pattern.

7. A (task statement 1.5) — Hooks are for deterministic guarantees where business rules demand compliance, or for reliable data transformation. Spelling preference carries no material consequence when occasionally missed and is well served by prompt guidance.

8. B (task statement 1.6) — Prompt chaining suits predictable multi-aspect reviews where the shape of the work is fixed regardless of content. Dynamic adaptive decomposition suits open-ended investigation where intermediate findings should change what happens next.

9. B (task statement 2.2) — The guide names the difference between retryable and non-retryable errors, and how returning structured metadata prevents wasted retry attempts. Status codes and stack traces still require interpretation, and asynchronous delivery does not address retry decisions.

10. D (task statement 2.2) — A policy violation is a business error that will not succeed on retry however many times it is attempted, so isRetryable must be false, and a customer-friendly explanation lets the agent communicate the outcome. Marking it transient invites pointless retries. A raw exception forces parsing. Returning empty-as-success is the suppression anti-pattern.

11. D (task statement 2.5) — The guide names using Read to load full file contents followed by Write when Edit cannot find unique anchor text. Repeating a failing Edit does not resolve non-uniqueness, and rewriting from memory risks losing content.

12. A (task statement 3.1) — The guide names using @import to selectively include relevant standards files in each package's CLAUDE.md based on maintainer domain knowledge. A single root file becomes a monolith. User-level files are not shared. Duplication creates drift.

13. D (task statement 3.6) — The guide names including prior review findings in context when re-running reviews after new commits, instructing Claude to report only new or still-unaddressed issues. Client-side diffing and file filtering do not give the model the awareness to judge whether an issue was addressed.

14. D (task statement 3.5) — The guide names providing specific test cases with example input and expected output to fix edge case handling, giving null values in migration scripts as the example. Prose remains interpretable, blanket defensiveness is unfocused, and a standing rule is disproportionate to one edge case.

15. D (task statement 4.5) — The guide names using prompt refinement on a sample set before batch-processing large volumes to maximise first-pass success rates and reduce iterative resubmission costs. Fixed batch sizes, warm-up runs and pre-validating documents against an output schema are not the described practice.

16. B (task statement 4.1) — The guide states that high false positive categories undermine confidence in accurate categories — trust in the review as a whole collapses. Cost, overfitting and build time are not the stated concern.

17. C (task statement 4.6) — The guide pairs per-file local analysis with a separate integration pass examining cross-file data flow. The per-file passes are already doing their job here; what is missing is the pass that looks across them. Loading every imported file into each per-file pass puts most of the change set back in front of every pass, which is the attention dilution the split existed to prevent. Asking the same session to infer the cross-file effects runs into the self-review limitation the guide describes: it retains the reasoning that produced those findings and is less likely to question them. Severity ranking decides what to look at again; it does not examine the relationships between files.

18. B (task statement 4.3) — The guide states directly that strict JSON schemas via tool use eliminate syntax errors but do not prevent semantic errors, giving line items that do not sum to the total as the example. Correct shape is not correct content.

19. A (task statement 5.6) — Task statement 5.6 names summarisation as where source attribution is lost: findings are compressed and the claim-source mapping is not among what survives, so synthesis receives claims with nothing attached to them. The remedy in the same task statement is structured claim-source mappings that every downstream step preserves and merges. Synthesis cannot restore a mapping it never received, differing citation formats would be a merge problem rather than a loss, and a missing template field would show as citations omitted from the report rather than claims whose sources are gone.

20. D (task statement 5.6) — The guide names completing document analysis with conflicting values included and explicitly annotated, letting the coordinator decide how to reconcile before passing to synthesis. Silent selection, nulling the field, or hidden reconciliation all destroy information the coordinator needs.